Ders 20 Genişletilmiş Notu: Uygulama 3: fork, exec, wait, Zombi ve Öksüz Süreçler ve Pipe ile Süreçler Arası İletişim

Ders 20 Genişletilmiş Notu: Uygulama 3: fork, exec, wait, Zombi ve Öksüz Süreçler ve Pipe ile Süreçler Arası İletişim

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Uygulamanın Yeri ve Akışı
- 2. Süreç Kavramının Tekrarı
- 3. Süreç Oluşturma: `fork()`
- 4. `fork` Örnekleri
- 5. `exec` Ailesi ve `execl` Örneği
- 6. `wait` ile Ebeveyni Bekletmek
- 7. Süreç Sonlandırma: `exit()` ve `atexit()`
- 8. Zombi ve Öksüz Süreçler
- 9. Çocuğun Çıkış Durumunu Okumak: `WIFEXITED` ve `WEXITSTATUS`
- 10. Süreçler Arası İletişim ve POSIX
- 11. Borular (Pipe)
- 12. Dosya Adı Gönderen İstemci-Sunucu Boru Örneği
- 13. Yarım Kalan Kısımlar
- Örnek Sorular
- Dersin Toparlaması

1. Uygulamanın Yeri ve Akışı

Bu kayıt, İşletim Sistemleri dersinin 3. uygulama (laboratuvar) oturumudur. Oturum Zoom üzerinden yapıldı. Dersin hocası, Dr. Ziya Cihan Tayşi, oturumu açıp uygulamayı yürütecek öğretim elemanına devretti ve ayrıldı. Asıl anlatım yaklaşık 00:13'te başlar. Öncesi katılımcıların toplanmasını beklemekle geçti.

Anlatıcı, bugün önce süreçlerden (process), sonra süreçler arası iletişimden (interprocess communication, IPC) bahsedileceğini söyledi. Süreç kısmının bir miktar tekrar olacağını, ama uygulamada kod üzerinden yapılacağı için bunun sorun olmayacağını belirtti. Bu kısımları hızlı geçeceğini, sorusu olanın sorabileceğini söyledi.

Oturum iki kez teknik sorunla kesildi. Yaklaşık 00:38 ile 00:47 arasında anlatıcının bağlantısı koptu. 01:12'den sonra ekran paylaşımı tamamen çöktü ve uygulama yarım kaldı. Bu yüzden FIFO ve sinyal kısımları anlatılamadı (bkz. Bölüm 13).

2. Süreç Kavramının Tekrarı

Süreç nedir? Bir program çalıştırılmıştır (run). Süreç, o programın o andaki görüntüsüdür. Bu görüntü birkaç açıdan birlikte düşünülür:

- bellekteki (memory) görüntüsü,

- işlemcideki (CPU) görüntüsü,
- kaynaklar (resource) açısından görüntüsü,
- durum (state) açısından görüntüsü: süreç devam mı ediyor, kesilmiş mi (interrupt), sonlanmış mı (terminate).

Programın bu tek görüntüsüne süreç denir.

Süreçler işletim sistemi tarafından takip edilir. Bunun nedeni, süreçlerin bellek ve işlemci gibi kaynakları ortak kullanmasıdır. İşletim sistemi süreçleri bir **süreç tablosunda (process table)** tutar. Bu tablonun her satırında bir sürecin bilgileri yer alır. Bu satıra süreç tanımlayıcı (process descriptor) ya da kontrol bloğu (process control block) denir. Satırda sürecin kimlik bilgileri (identifier), durumu ve kaynakları yazılıdır. Satır ayrıca sürecin koduna ve verisine de referans verir.

Anlatıcı slayttaki bir şekil üzerinden, bir kontrol bloğunda neler tutulduğunu sıraladı:

- sürecin nerede olduğu (o anki çalışma dizini, current directory),
- argümanları,
- değişkenleri (ortam değişkenleri, environment variables),
- sürece gelen ya da gelecek sinyaller (signal),
- kullandığı dosyalar (file),
- hangi kodu çalıştırdığı,
- hangi veriyi kullandığı.

Slayttaki örnek süreç, bir dizinin altında hangi dizin ve dosyaların olduğuna bakan basit bir `ls -l` komutuydu. Çalışma dizini bir kullanıcının ev dizini olarak gösterilmişti. Argüman listesinde sıfırıncı argüman, birinci argüman diye devam eden öğeler vardı. Değişken listesinde kullanıcı kimliği (user id) gibi ortama dair her şey tutuluyordu. Ardından sinyaller, açık dosyalar ve kodun yeri geliyordu.

3. Süreç Oluşturma: `fork()`

Süreç oluşturmak için iki sistem çağrısı (system call) kullanılır: `fork` ve `exec`. Anlatıcı farkı şöyle özetledi: `fork`, sürecin kendisinin aynısının bir kopyası olan bir çocuk (child) süreç oluşturur. `exec` ile ise çocuk bambaşka bir iş yapabilir.

`fork` sonrasında ebeveyn (parent) ile çocuk, süreç kimliği (PID) dışında her şeyiyle birbirinin aynısıdır. Önemli nokta şudur: Çocuk, kodun `fork()`'tan **sonraki** kısmından devam eder. Kod üzerinde şöyle düşünülebilir: Ebeveyn kodun tamamını çalıştırır, çocuk ise `fork()`'tan sonrasını çalıştırır.

`fork()`'un dönüş değeri ebeveyn ile çocuğu ayırt etmeyi sağlar:

- Ebeveynde `fork()`, oluşturulan çocuğun PID'sini döndürür.
- Çocukta `fork()`'un dönüş değeri 0'dır.
- Çocuk kendi PID'sini öğrenmek isterse `getpid()` çağırmalıdır.

Yani `fork()`'un döndürdüğü değer bir değişkende tutulursa, bu değişken ebeveynde çocuğun PID'sini, çocukta 0'ı gösterir.

> **Ek bilgi (derste anlatılmadı):** `fork()` başarısız olursa (örneğin süreç sınırı dolmuşsa) çocuk oluşturulmaz ve ebeveyne -1 döner. Bu yüzden gerçek programlarda dönüş değeri üç duruma göre kontrol edilir: -1 hata, 0 çocuk, pozitif değer ebeveyn.

4. `fork` Örnekleri

Anlatıcı uygulama klasöründeki örnek programları derleyip çalıştırdı.

4.1 Birinci Örnek: `fork`'tan Önce ve Sonra

İlk örnek basit bir programdı. `fork()`'tan önce bir mesaj ("before fork"), `fork()`'tan sonra bir mesaj ("after fork") yazıyordu. Anlatıcı önce öğrencilere tahminlerini sordu, sonra tahmini birlikte kurdu:

- `fork()`'tan önceki mesaj yalnız ebeveynde, bir kez basılır.

- `fork()` çağrılınca süreç kendini kopyalar.
- `fork()`'tan sonraki mesaj bir kez çocukta, bir kez ebeveynde, yani iki kez basılır.

Çalıştırınca da böyle oldu. İlk "before fork" satırı ebeveynden geldi. Sonraki iki "after fork" satırından hangisinin önce çocuktan, hangisinin ebeveynden geldiği ise işletim sisteminin hangisini önce çalıştırdığına bağlıdır. Anlatıcı buna yarış durumu (race condition) dedi. Her iki sıra da mümkündür.

4.2 İkinci Örnek: Kimin Yazdığını Görmek

İkinci örnekte (`fork2`) her süreç kendi PID'sini ve `fork()`'un sonucunu yazıyordu. Program önce `getpid()` ile kendi PID'sini yazar. Sonra `fork()` çağırır ve sonucu bir değişkende tutar. Ardından iki süreç de "process id : <getpid()> - result : " biçiminde yazar. Derste görülen çıktı şuydu:

```
process id : 4917
process id : 4918 - result : 0
process id : 4917 - result : 4918
```

Satırlar şöyle okunur:

- İlk satır `fork()`'tan önce, ebeveynde yazılmıştır. Ebeveynin PID'si 4917'dir.
- İkinci satırda sonuç 0'dır. Demek ki bu satırı çocuk yazmıştır ve bu çalıştırmada önce çocuk çalışmıştır. Çocuk `getpid()` ile kendi PID'sini (4918) bulmuştur. `fork()` sonucu ise 0'dır, çünkü çocuk `fork()`'tan sonrasını çalıştırır ve değişkene çocuğun PID'si yalnız ebeveynde yazılır.
- Üçüncü satır ebeveyne aittir. `getpid()` yine 4917'dir. `fork()` sonucu, oluşturulan çocuğun PID'si olan 4918'dir.

Anlatım sırasında bir an "çocukta `getpid` 0 oldu" dendi, anlatıcı hemen düzeltti: `getpid()` 4918'dir, 0 olan `fork()`'un sonucudur.

4.3 Üçüncü Örnek: Ebeveyn ve Çocuğa Farklı İş Yaptırmak

Üçüncü örnekte `fork()`'un sonucu bir `if-else` içinde kullanılıyordu. Anlatıcı önce sınıfa sordu: `fork()` sıfır dönerse ne oluyordu? Cevap: Çocuk sürecin içindeyiz demektir. C'de `if` içinde sıfır dışındaki her değer doğru (true) sayılır, yalnız sıfır `else`'e düşer.

Program önce kendi PID'sini aldı (derste 4996). Sonra `fork()` ile bir çocuk oluşturdu. Ebeveynde `fork()` sıfır dışında döndüğü için `if` kısmına girildi ve yalnız ebeveynin işi yapıldı. Çocukta sonuç sıfır olduğu için `else` kısmına girildi ve yalnız çocuğun işi yapıldı. Bu yüzden iki dal da birer kez çalıştı. `fork()`'tan önceki satır da yalnız ebeveynde bir kez çalıştı.

4.4 Neden Farklı İşler?

Anlatıcı burada bir soru sordu: İki süreci neden oluşturduk? Paralel olarak bir şeyler yürüsün diye. Süreci tamamen aynısıyla kopyalarsak aynı iş iki kez yürür. Bunun kullanıldığı yerler vardır, ama genelde farklı davranan iki sürecin aynı anda yürümesi istenir. Bunun iki yolu vardır:

- üçüncü örnekteki gibi `if-else` ile ebeveyne bir iş, çocuğa başka bir iş yaptırmak,
- `exec` kullanmak.

4.5 Dördüncü Örnek: Alıştırma Sorusu (`fork4`)

`exec`'e geçmeden önce slayttaki alıştırma sorusu (practice question) çözüldü. Kod, kısa notta ekran karesinden alınan biçimiyle şöyledir:

```c
#include <stdio.h>
#include <unistd.h>

int main() {
    if (fork()) {
        if (!fork()) {
```

```
            fork();
            printf("1 \n");
        } else {
            printf("2 \n");
        }
    } else {
        printf("3 \n");
    }
    printf("4 \n");
    return 0;
}
```

Kodda üç `fork()` çağrısı vardır. Anlatıcı süreç ağacını bir aile benzetmesiyle anlattı (büyükbaba, baba, amca, amca çocuğu), ama benzetme karışık kaldı (bkz. Ders sonrası not). Ağaç aşağıdaki gibi kurulur:

- **Ana süreç (P):** İlk `fork()` ebeveynde sıfır dışında döner, `if`'e girer. İkinci `fork()`'u çağırır. Bu da ebeveynde sıfır dışında döner, yani `!fork()` yanlıştır ve `else`'e düşer. "2" yazar. Dıştaki yapıdan çıkar ve "4" yazar.
- **P'nin ilk çocuğu (C1):** İlk `fork()` bu süreçte 0 döner. Dıştaki `else`'e düşer. "3" yazar, sonra "4" yazar.
- **P'nin ikinci çocuğu (C2):** İkinci `fork()` bu süreçte 0 döner, `!fork()` doğrudur. İçeri girer ve üçüncü `fork()`'u çağırır. "1" yazar, sonra "4" yazar.
- **C2'nin çocuğu (C3):** Üçüncü `fork()`'tan sonra C2 ile aynı yerden devam eder. "1" yazar, sonra "4" yazar.

Sonuç: toplam 4 süreç vardır. "1" iki kez, "2" bir kez, "3" bir kez, "4" dört kez yazılır.

Anlatıcı, satırların sırasının işletim sistemi tarafından belirlendiğini, bu yüzden farklı sıraların görülebileceğini, bunu önlemenin yolunun `wait` olduğunu söyledi. Derste görülen sıra şuydu:

```
2
4
3
4
1
4
1
4
```

Sırayla: ebeveyn (P) "2" ve "4" yazdı. Sonra ilk çocuk (C1) "3" ve "4" yazdı. Sonra C2 "1" ve "4" yazdı. En son C3 "1" ve "4" yazdı. Anlatıcı, bunu tahtada çizmediği için gözde canlandırmanın zor olabileceğini söyledi ve soru olup olmadığını sordu.

> **Ders sonrası not:** Program `gcc -Wall` ile derlenip çalıştırıldı. "1" iki kez, "2" ve "3" birer kez, "4" dört kez yazıldı. Derste anlatılan sonuçla aynıdır.

5. exec Ailesi ve `execl` Örneği

`exec` tek bir fonksiyon değil, bir fonksiyon ailesidir (exec family functions). Anlatıcı slayttaki listeyi gösterdi: Kimi üyeler argümanları ayrı ayrı alır, kimi bir dizi (array) içinde alır, ayrıca başka çeşitleri de vardır. Ayrıntılar için kılavuz sayfalarına (man page) bakılabileceğini söyledi. Kendisi en temel olan `execl` ile bir örnek hazırlamıştı.

> **Ek bilgi (derste anlatılmadı):** Ad içindeki harfler argümanların nasıl verildiğini gösterir. `l` (list): argümanlar `execl(yol, arg0, arg1, ..., NULL)` gibi tek tek verilir. `v` (vector): argümanlar `NULL` ile biten bir dizi olarak verilir. `p` (path): program adı `PATH` ortam değişkenindeki dizinlerde aranır. `e` (environment): yeni programa ayrı bir ortam değişkenleri dizisi verilir.

Örnek kod (`example 5`), kısa notta ekran karesinden alınan biçimiyle şöyledir (ev dizini yolu genelleştirildi):

```
int pid;
printf("Parent does stuff and then calls fork...\n");
```

```
pid = fork();
if (pid) {                      // ebeveyn
    // wait(pid);
    printf("... parent do something completely different\n");
} else {                        // çocuk
    printf("Child runs an executable...\n");
    execl("/bin/ls", "/bin/ls", "-l", "/home/<kullanıcı>/Desktop/", NULL);
}
exit(0);
```

Ebeveyn önce bir mesaj yazar, sonra `fork()` çağırır. Ebeveyn `if` dalında kendi mesajını yazar. Çocuk `else` dalında bir mesaj yazar ve `execl` ile `/bin/ls -l` programını çalıştırır. `execl`'in ilk argümanı çalıştırılacak dosyanın yoludur. İkincisi programın sıfırıncı argümanıdır (geleneksel olarak programın adı). Sonra `-l` seçeneği ve listelenecek dizin gelir. Liste `NULL` ile biter.

Çalıştırınca çocuk masaüstündeki dosyaları `ls -l` biçiminde listeledi. Anlatıcının ekranda yorumuna göre bu çalıştırmada önce çocuk çalıştı, çıkış durumunu döndürdü, ardından ebeveyn kendi mesajını yazdı.

> **Ders sonrası not:** Derste `exec` için "child process'in kendi kendine bir şeyler yapma hakkı var" ve "bu child bunun içinde bir tane daha child oluşturmuş olacak, `ls`'i çalıştıran bir çocuk oluyor" dendi; çıktı da "child'ın child'ı" gibi yorumlandı. Bu yanlıştır. `exec` yeni süreç oluşturmaz. Çağıran sürecin kodunu ve verisini yeni programla değiştirir; PID aynı kalır. Örnekte `ls`'i çalıştıran süreç, `fork` ile oluşturulan çocuğun kendisidir. `execl` başarılı olursa çocukta sonraki satırlar (`exit(0)`) hiç çalışmaz. Bu, kod `execl` sonrasına bir `printf` eklenip derlenerek denendi: o satır hiç basılmadı.

6. `wait` ile Ebeveyni Bekletmek

Anlatıcı ardından aynı örnekte ebeveynin önce çocuğu beklemesini istedi: Çocuk işini bitirsin, ebeveyn onu beklesin, sonra kendi işini yapsın. Bunun için koddaki yorum satırına alınmış bekleme satırı açıldı ve program yeniden çalıştırıldı. Bu kez akış şöyleydi: Ebeveyn önce `fork`'tan önceki mesajı yazdı. `fork` çağrıldı. Ebeveyn bekledi, çocuğun içindeki işler (`ls` listesi) yapıldı. Çocuk bitince ebeveyn kendi mesajını yazdı. Yani ebeveyn, PID'si verilen çocuğun işini bitirmesini bekledi, sonra kendi işlemlerini yaptı.

Anlatıcı sonra şunu sordu: Ebeveynin, önceki ağaçtaki gibi onlarca çocuğu olsaydı ne olurdu? Bu durumda PID yerine `NULL` yazılabileceğini, tek çocuk olduğu için PID vererek de aynı şeyin yapılabileceğini söyledi.

> **Ders sonrası not:** Koddaki `wait(pid)` doğru bir çağrı değildir. `wait`'in parametresi çocuğun çıkış durumunun yazılacağı bir `int *`'dır, PID değildir. Belli bir çocuğu beklemek için `waitpid(pid, NULL, 0)`, herhangi bir çocuğu beklemek için `wait(NULL)` yazılır. Derste "`NULL` yazarsak bütün çocuklar bittikten sonra ebeveyn kendi işini yapar" dendi; bu da yanlıştır. `wait(NULL)` yalnız **herhangi bir** çocuk sonlanana kadar bekler. Bütün çocukları beklemek için `while (wait(NULL) > 0);` döngüsü kullanılır. Örnek `waitpid(pid, NULL, 0)` ile derlenip çalıştırıldı; ebeveynin mesajı `ls` çıktısından sonra geldi.

Bu noktada (yaklaşık 00:38) anlatıcının bağlantısı koptu. 00:47'de geri döndüğünde ekran paylaşımı sorunluydu. Ekranı göremediği için kalan kısmı slaytlar ve kod dosyaları üzerinden, çoğunu çalıştırmadan anlattı.

7. Süreç Sonlandırma: `exit()` ve `atexit()`

Süreç sonlandırmada (process termination) `exit()` fonksiyonuna bir tamsayı durum (status) verilir. Bütün işlerini yapmış ve normal şekilde sonlanan bir süreç için buraya 0 yazılır, yani durum 0 olur. Bunun dışındaki değerler, örneğin bir hata oluştuğunu bildirmek için kullanılır.

`atexit()` ile bir fonksiyona referans verilir. Bu fonksiyon, sürecin çıkışında çağrılır. Anlatıcı bunu "nasıl ki sıfır döndürebiliyor, burada da bu fonksiyonu döndürebiliyor" diye anlattı. Slayttaki örnekte `parent cleaner` adlı bir fonksiyon yazılmış ve `atexit` ile kaydedilmişti. Ekrandaki çıktıda ebeveyn "this is parent" yazıyor, çıkışta da temizleyici fonksiyon çalışıyordu.

> **Ders sonrası not:** Derste `atexit` için "çocuk süreç bittiği zaman parent cleaner'ı çağıracağız" dendi. Doğrusu: `atexit` ile kaydedilen fonksiyon, onu kaydeden süreç normal sonlanırken (`exit` ya da `main`'den `return`) çağrılır. Çocuğun sonlanmasıyla ilgisi yoktur. Kayıt `fork`'tan önce yapılırsa çocuk da bu kaydı devralır ve kendi çıkışında aynı fonksiyonu çağırır. Bu, kayıt `fork`'tan önce yapılan küçük bir programla denendi: temizleyici fonksiyon hem çocukta hem ebeveynde, her biri kendi PID'siyle bir kez çalıştı.

8. Zombi ve Öksüz Süreçler

Anlatıcı, `wait` örneğine geri bağlanarak süreçler birbirini beklemezse bazı sorunlar çıktığını söyledi. Bunlar zombi (zombie) ve öksüz (orphan) süreçlerdir; bazı kaynaklar bunları ikiye ayırır.

Doğru akış şöyledir: Çocuk sonlanınca çıkış durumunu (termination status) ebeveynine döndürür. Ebeveyn bunu kabul eder. İşletim sistemi de çocuğu süreç tablosundan çıkarır.

Zombi süreç, sonlanma durumunda (termination state) olduğu hâlde süreç tablosunda hâlâ yer işgal eden süreçtir. Çıkış durumunu vermiştir, devam etmemesi gerekir, ama tablodaki kaydı silinmemiştir. Bunun nedeni, ebeveynin çocuğun çıkış durumunu görüp onaylamamasıdır.

Zombi süreçler `ps aux` çıktısında görülebilir. Durum (STAT) sütununda büyük `Z` harfi yazar. Çıktı `grep Z` ile süzülerek zombi süreçler bulunabilir. Anlatıcı onları öldürmenin birkaç yolunu saydı: `kill` ile, ya da ebeveyn sürece bütün çocuklarını öldürmesi için sinyal göndererek. Bunun her zaman işe yaramadığını, ebeveyn düzgün kodlanmamışsa `kill -9` ile kesilebileceğini, `kill -9` sinyalinin yakalanamayacağını ve yok sayılamayacağını (ignore) söyledi. Sinyallere sonra gelineceğini belirtti.

Slayttaki iki örnek şöyle anlatıldı:

- **Zombi örneği:** Ebeveyn sonsuz döngüdedir ve çocuğu beklemez. Çocuk çıktığı hâlde ebeveyn onun bittiğini, çıkış yaptığını göremez. Çocuk zombi olarak kalır.
- **Öksüz örneği:** Bu kez çocuk sonsuz döngüdedir. Ebeveyn ondan önce çıkıp gitmiştir. Çocuk öksüz kalır.

Ebeveyn ile çocuk arasındaki bu eşzamanlamayı (synchronization) sağlamak için `wait` sistem çağrısı kullanılır.

> **Ders sonrası not:** Derste "ebeveyn, çocuk beklemedeyken sonlandırılırsa yine zombi ortaya çıkıyor, aslında ona orphan da denebiliyor" dendi; burada zombi ile öksüz karıştırıldı. Ebeveyni çocuktan önce sonlanan süreç öksüzdür ve zombi değildir. Öksüz süreç, init/systemd ya da sistemdeki bir alt toplayıcı (subreaper) süreç tarafından evlat edinilir. Zombi ise `kill -9` ile öldürülemez, çünkü zaten sonlanmıştır. Çözüm ebeveynin `wait` çağırmasıdır ya da ebeveyni sonlandırmaktır. Ebeveyn sonlanınca zombi yeni ebeveynine geçer ve o tarafından temizlenir. İki durum küçük programlarla denendi: Ebeveyn `wait` çağırmadan uyurken çocuk `ps` çıktısında `Z` durumunda göründü. Ebeveyni önce sonlanan çocukta `getppid()` başka bir PID gösterdi. Deneme süreçleri sonra sonlandı.

9. Çocuğun Çıkış Durumunu Okumak: `WIFEXITED` ve `WEXITSTATUS`

Çocuk sonlanma durumuna geldiğinde çıkış durumunu ebeveyne gönderir. Ebeveyn bunu bir değişkende tutar (slayttaki adı `child status`). Bu değer iki makroyla incelenir:

- `WIFEXITED(durum)`: Bu durum oluştuysa çocuk normal bir şekilde çıkış yapmış demektir.
- `WEXITSTATUS(durum)`: Çocuğun çıkış değerini (exit status) verir.

Anlatıcı, çıkış değeri özelleştirilmiş bir değer değilse ve 0 döndüyse çocuğun normal şekilde sonlandığını, değilse anormal (abnormal) sonlandırıldığının söylenebileceğini belirtti.

> **Ders sonrası not:** Kayıt `fork`'tan önce yapılan bir `atexit` denemesinde çocuk `exit(3)` ile sonlandırıldı; ebeveynde `wait(&durum)` sonrası `WIFEXITED` doğru, `WEXITSTATUS` 3 çıktı. Yani sıfır dışında bir çıkış değeri de "normal sonlanma" sayılır;

`WIFEXITED` sürecin `exit` ile mi yoksa bir sinyalle mi bittiğini ayırır, değerin 0 olup olmadığını ise `WEXITSTATUS` gösterir.

10. Süreçler Arası İletişim ve POSIX

Süreçler arası iletişim iki çeşit olabilir:

- **Borular (pipe):** Ebeveyn ile çocuk arasında kullanılır.
- **Adlandırılmış borular (named pipe, FIFO):** Birbirinin ebeveyni ya da çocuğu olmayan farklı süreçler arasında da iletişim sağlar.

Anlatıcı burada POSIX'ten de söz etti. POSIX'i sinyallerle ilgili bir standartlar bütünü olarak tanıttı ve bir kütüphane gibi düşünülebileceğini söyledi. UNIX sistemlerde kullanılan sinyallerin bu standartlara göre yapıldığını belirtti. Ayrıca kullanıcının içini kendisinin doldurduğu, fonksiyonunu kendisinin yazdığı yapılar (structure) da olduğunu, bunların sinyaller üzerinde kullanıcıya da hak tanımak için bulunduğunu söyledi.

> **Ders sonrası not:** POSIX (Portable Operating System Interface) yalnız sinyallerle ilgili bir standart değildir. UNIX benzeri işletim sistemlerinin programlama arayüzünü tanımlayan geniş bir standartlar ailesidir. Sinyaller, borular, `fork`, `exec`, `wait` ve Pthreads bunun parçasıdır.

11. Borular (Pipe)

Borular tek yönlüdür. Linux'te kabukta bilinen `|` işareti de bir borudur. `komut1 | komut2 | komut3` biçiminde kullanıldığında veri soldan sağa akar: Soldakinin çıktısı sağdakinin girdisi olur, onun çıktısı da bir sonrakinin girdisi olur. Anlatıcı ayrıca < ve > işaretleriyle de benzer yönlendirme yapılabileceğini söyledi.

11.1 Basit Boru Örneği (`Ex_1_pipe1.c`)

İlk örnekte boru üzerinden tek bir tamsayı gönderiliyordu. Kod, kısa notta ekran karesinden alınan biçimiyle şöyledir:

```c
int fd[2];               // fd[0] okuma ucu, fd[1] yazma ucu
if (pipe(fd) == -1) {
    printf("An error occured with oppening pipe");
    return 1;
}
int childpid = fork();
if (childpid == 0) {     // çocuk: yazar
    close(fd[0]);
    int x;
    printf("Enter a number: ");
    scanf("%d", &x);
    write(fd[1], &x, sizeof(int));
    close(fd[1]);
} else {                 // ebeveyn: okur
    int y;
    close(fd[1]);
    read(fd[0], &y, sizeof(int));
    close(fd[0]);
    printf("Child with PID: %d sent %d \n", childpid, y);
}
```

Anlatıcı kodu adım adım açıkladı:

- Borunun iki ucu bir dizide tanımlanır. `fd[0]` borunun okuma ucu (read end), `fd[1]` yazma ucudur (write end).
- `pipe(fd)` boruyu oluşturur. Oluşturulamazsa -1 döner ve bir hata mesajı yazılıp çıkılır.
- Sonra `fork()` ile bir çocuk oluşturulur.

Burada önemli bir nokta vardır. `fork` sürecin birebir kopyasını oluşturduğu için borunun iki ucu da hem ebeveynde hem çocukta açıktır. Anlatıcı `fd`'yi bir dosya işaretçisi (file pointer) gibi düşünmenin de fark etmeyeceğini söyledi. Çocuk yalnız yazacaksa, en başta okuma ucunu kapatabilir.

- **Çocuk (yazan):** Okuma ucunu (`fd[0]`) kapatır. `x` değişkenini tanımlar, kullanıcıdan bir sayı alır, bu sayıyı yazma ucuna (`fd[1]`) yazar. İşi bitince yazma ucunu da kapatır.
- **Ebeveyn (okuyan):** Kendi `y` değişkenini tanımlar. Bir şey yazmayacağı için yazma ucunu (`fd[1]`) kapatır. Okuma ucundan bir tamsayı okur, okuma bitince okuma ucunu kapatır ve sonucu ekrana yazar.

  Hocanın vurgusu: Boru tek yönlüdür; buna half-duplex denir. Anlatıcı bunun sınavda çıkmış olabileceğini söyledi.

Çalıştırmada önce "Enter a number" iletisi beklenen şekilde görünmedi. Anlatıcı yanlış bir şey yaptığını söyleyip yeniden çalıştırdı. Sonra 5 girildi. "Enter a number" iletisi çocuktaydı: Çocuk kullanıcıdan sayıyı istedi ve boruya yazdı. Ebeveyn sayıyı borudan okudu ve "Child with PID: … sent 5" yazdı.

> **Ders sonrası not:** Altyazıdaki "has duplex" ifadesi "half-duplex" demektir. Sıradan (anonim) boru tek yönlüdür; iki yönlü iletişim için iki boru gerekir. Ayrıca ekrandaki kodda `#include <unistd.h>` eksiktir (`pipe`, `fork`, `read`, `write`, `close` için gerekir). GCC 15 bu eksik bildirimi uyarı değil hata olarak verir. Başlık eklenip `gcc -Wall` ile derlenen program, girdi olarak 5 verilince "Child with PID: … sent 5" yazdı.
>
> **Ek bilgi (derste anlatılmadı):** Kullanılmayan uçları kapatmanın pratik bir nedeni vardır. Okuyan taraf, borunun bütün yazma uçları kapanınca `read` çağrısından 0 alır ve verinin bittiğini anlar. Okuyan taraf kendi yazma ucunu açık bırakırsa bu "dosya sonu" bilgisi hiç gelmez ve okuma sonsuza kadar bekleyebilir.

12. Dosya Adı Gönderen İstemci-Sunucu Boru Örneği

İkinci örnek, basit borudan farklı olarak bir dosyadan okuyup yazıyordu. Yine borunun iki ucu tanımlanıyordu. Program kullanıcıdan bir dosya adı istiyordu; anlatıcı ekrana "Enter file name" yazdırmayı unuttuğunu söyledi.

Roller şöyleydi:

- **İstemci (client, ebeveyn):** Kullanıcıdan dosya adını alır ve bunu tampona (buffer), oradan boruya yazar.
- **Sunucu (server, çocuk):** Yazma ucunu kapatır ve okuma ucunu kullanır. Bir tampon tanımlar. Borudan dosya adını okur. Bu adla dosyayı salt okunur olarak açar. Açamazsa "can't open" iletisi verir. Açabilirse dosyanın içeriğini tampona okur ve standart çıktıya (`STDOUT_FILENO`) tampondaki `n` bayt kadar veriyi yazar.

Anlatıcı önce rolleri karıştırdı, sonra düzeltti: İstemci hizmet isteyen taraftır ve dosya adını söyler; sunucu hizmeti sağlayan taraftır ve dosyayı açar. Klasörde bu örnek için hazırlanmış bir dosya da vardı.

13. Yarım Kalan Kısımlar

01:12 civarında ekran paylaşımı tamamen çöktü. Öğrenciler yalnız anlatıcının kamerasını görebildiklerini söyledi. Anlatıcı, uygulamanın kalanı için hocanın izniyle telafi yapacağını söyledi. Kalan konular şunlardı:

- borular baştan tekrar alınacak,
- adlandırılmış borular (FIFO) ve bunlarla ilgili bir iki örnek,
- sinyaller ve sinyal eylemleri (`sigaction`).

Telafi ya ayrı bir akşam oturumunda çevrim içi yapılacak ya da bir sonraki uygulamanın ilk yarım saatinde yapılacaktı. Duyuru yapılacağı söylendi. Slaytların paylaşılacağı da belirtildi.

Örnek Sorular

Soru 1 (derste sözlü, 00:19:45): `fork()`'tan önce "before fork", `fork()`'tan sonra "after fork" yazan bir programda hangi mesaj kaç kez basılır?

Çözüm (derste verildi): "before fork" yalnız ebeveynde bir kez basılır. `fork()` süreci kopyalar ve çocuk `fork()`'tan sonrasını çalıştırır. Bu yüzden "after fork" iki kez basılır: bir kez ebeveynde, bir kez çocukta. Bu iki satırdan hangisinin önce geleceği işletim sisteminin zamanlamasına bağlıdır.

Soru 2 (derste sözlü, 00:23:21): `fork()`'un sonucu sıfır gelirse hangi süreçteyiz?

Çözüm (derste verildi): Çocuk süreçteyiz. Ebeveynde `fork()` çocuğun PID'sini döndürür. C'de `if (fork())` yazıldığında sıfır dışındaki her değer doğru sayılır; yalnız sıfır, yani çocuk, `else` dalına düşer.

Soru 3 (derste sözlü, alıştırma sorusu, 00:27:09): Bölüm 4.5'teki `fork4` programında kaç süreç oluşur ve ekrana hangi satırlar kaç kez basılır?

Çözüm (derste verildi): Üç `fork()` çağrısıyla ana süreçle birlikte toplam 4 süreç olur. Ana süreç "2" ve "4", ilk çocuğu "3" ve "4", ikinci çocuğu "1" ve "4", ikinci çocuğun çocuğu "1" ve "4" yazar. Toplamda "1" iki kez, "2" bir kez, "3" bir kez, "4" dört kez basılır. Sıra işletim sistemine bağlıdır; derste `2 4 3 4 1 4 1 4` görüldü. Sonuç `gcc -Wall` ile derlenip çalıştırılarak doğrulandı.

Dersin Toparlaması

Bu uygulamada süreç kavramı kod üzerinden tekrar edildi. Süreç, çalışan bir programın bellek, işlemci, kaynak ve durum açısından o anki görüntüsüdür ve işletim sistemi onu süreç tablosunda bir kontrol bloğuyla tutar. `fork()` sürecin kopyasını oluşturur; çocuk `fork()`'tan sonrasını çalıştırır ve dönüş değeri ebeveynde çocuğun PID'si, çocukta 0'dır. Bu dönüş değeriyle ebeveyne ve çocuğa farklı işler yaptırılabilir ya da `exec` ailesiyle çocuğun içeriği tamamen başka bir programla değiştirilebilir. Çalışma sırası işletim sistemine bağlı olduğu için ebeveynin çocuğu beklemesi gerekiyorsa `wait`/`waitpid` kullanılır. Ebeveyn çocuğun çıkış durumunu almazsa zombi, çocuktan önce sonlanırsa öksüz süreç ortaya çıkar.

Dersin ikinci kısmında süreçler arası iletişime geçildi. Boru, ebeveyn ile çocuk arasında tek yönlü (half-duplex) bir kanaldır: `fd[0]` okuma, `fd[1]` yazma ucudur ve her taraf kullanmadığı ucu baştan kapatır. Teknik sorunlar yüzünden dosya adı gönderen istemci-sunucu örneği tamamlanamadı; adlandırılmış borular (FIFO) ve sinyaller (`sigaction`) telafi oturumuna kaldı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.